

datos masivos desde archivos CSV utilizando tablas temporales intermedias para la limpieza de datos (casting de tipos, manejo de nulos).

2. Parte 1: Explotación de Datos (SQL):

Se diseñaron 11 consultas SQL complejas para extraer información de negocio relevante (estadísticas de pilotos, escuderías, circuitos). Se utilizaron técnicas como JOIN para combinar tablas, funciones de agregación (COUNT, SUM, MAX, MIN) y subconsultas estándar para resolver problemas de filtrado avanzado (como encontrar máximos o mínimos absolutos) sin recurrir a sentencias no permitidas como LIMIT o funciones de ventana (RANK).

3. Parte 2: Automatización y Seguridad (PL/pgSQL):

- **Automatización:** Se implementaron *Triggers* para auditar cambios en los datos (tabla auditoria_eventos) y para mantener calculados en tiempo real los puntos totales de los pilotos (tabla puntuacion_total_pilotos), mejorando el rendimiento de lectura.
- **Seguridad:** Se definió un esquema de roles jerárquico (Administrador, Gestor, Analista, Invitado) con permisos granulares (GRANT/REVOKE) para cumplir con el principio de mínimo privilegio. Se aseguró que la Vista creada en la Parte 1 estuviera disponible antes de asignar permisos.

4. Parte 3: Integración (Python):

Se desarrolló una aplicación de consola en Python (Interfaz.py) que utiliza la librería psycopg2 para conectarse a PostgreSQL. La aplicación permite iniciar sesión con los diferentes roles creados y, dependiendo de sus permisos, ejecutar consultas de lectura o insertar nuevos datos operativos (Carreras y Resultados), manejando adecuadamente los errores de permisos.

IV. PARTE 1: CONSULTAS A LA BASE DE DATOS

Consulta 1: Listar Circuitos y Número de Grandes Premios

- **Descripción:** Cuenta cuántas carreras (races) se han celebrado en cada circuito, agrupando por el identificador del circuito y ordenando de mayor a menor frecuencia.
- **Código SQL:**
- SELECT c.name AS "Circuito", COUNT(r.name) AS "Numero_de_GPs"
- FROM pl1final.circuits c
- JOIN pl1final.races r ON c.circuitRef = r.circuitRef

- GROUP BY c.circuitRef, c.name
- ORDER BY "Numero_de_GPs" DESC;

- Resultado (Muestra):

| Circuito | Numero_de_GPs |

| :--- | :--- |

| Autodromo Nazionale di Monza | 73 |

| Circuit de Monaco | 69 |

| Silverstone Circuit | 58 |

Consulta 2: GPs y Puntos de 'Ayrton Senna'

- **Descripción:** Filtra los resultados del piloto 'Ayrton Senna', contando sus participaciones y sumando sus puntos totales.
- **Código SQL:**
- SELECT COUNT(res.name) AS "GPs_Corridos_Senna", SUM(res.Puntos) AS "Puntos_Totales_Senna"
- FROM pl1final.results res
- JOIN pl1final.drivers d ON res.driverRef = d.driverRef
- WHERE d.forename = 'Ayrton' AND d.surname = 'Senna';

- Resultado:

| GPs_Corridos_Senna | Puntos_Totales_Senna |

| :--- | :--- |

| 162 | 614 |

Consulta 3: Pilotos nacidos después de 1999

- **Descripción:** Selecciona pilotos jóvenes (nacidos > 31/12/1999) y cuenta sus carreras. Se usa LEFT JOIN para incluir a aquellos que, aun cumpliendo el criterio de edad, no han participado en ninguna carrera (resultado 0).
- **Código SQL:**
- SELECT d.forename AS "Nombre", d.surname AS "Apellido", COUNT(res.name) AS "Carreras_Participadas"

- FROM pl1final.drivers d
- LEFT JOIN pl1final.results res ON d.driverRef = res.driverRef
- WHERE d.dob > '1999-12-31'
- GROUP BY d.driverRef, d.forename, d.surname
- ORDER BY "Carreras_Participadas" DESC;
- Resultado (Muestra):

| Nombre | Apellido | Carreras_Participadas |

| :--- | :--- | :--- |

| Yuki | Tsunoda | 63 |

| Oscar | Piastri | 20 |

Consulta 6: Ganadores (max puntos) de las temporadas 2010-2015

- **Descripción:** Utiliza la Vista creada en la Consulta 5. Para encontrar al ganador (máximo de puntos) sin usar funciones de ventana, se emplea una **subconsulta en el JOIN** que calcula el máximo de puntos por temporada y se cruza con la tabla original.
- **Código SQL:**
- SELECT v."Temporada", v."Nombre_Piloto", v."Apellido_Piloto",
v."Puntos_Totales_Temporada"
- FROM pl1final.VistaPuntosPorTemporada v
- JOIN (
- -- Subconsulta: Calcula el máximo de puntos por cada temporada
- SELECT "Temporada", MAX("Puntos_Totales_Temporada") as
"Max_Puntos"
- FROM pl1final.VistaPuntosPorTemporada
- WHERE "Temporada" BETWEEN 2010 AND 2015
- GROUP BY "Temporada"
-) mp ON v."Temporada" = mp."Temporada"
- AND v."Puntos_Totales_Temporada" = mp."Max_Puntos"
- ORDER BY v."Temporada" ASC;

- Resultado (Muestra):

| Temporada | Nombre_Piloto | Apellido_Piloto | Puntos |

| :--- | :--- | :--- | :--- |

| 2010 | Sebastian | Vettel | 256 |

| 2011 | Sebastian | Vettel | 392 |

Consulta 9: Piloto con la vuelta más rápida histórica

- **Descripción:** Busca al piloto que posee el tiempo de vuelta mínimo absoluto. Se utiliza una **subconsulta en el WHERE** para encontrar ese valor mínimo (MIN("time")) y filtrar la tabla principal, evitando el uso prohibido de LIMIT.
- **Código SQL:**
 - SELECT d.forename AS "Nombre", d.surname AS "Apellido", lt."time" AS "Vuelta_Rapida_Historica"
 - FROM pl1final.lap_times lt
 - JOIN pl1final.drivers d ON lt.driverRef = d.driverRef
 - WHERE lt."time" = (
 - -- Subconsulta: Encuentra el tiempo mínimo absoluto de toda la tabla
 - SELECT MIN("time")
 - FROM pl1final.lap_times
 -);

- Resultado:

| Nombre | Apellido | Vuelta_Rapida_Historica |

| :--- | :--- | :--- |

| George | Russell | 00:55.404 |

(El resto de consultas 4, 5, 7, 8, 10 y 11 siguen la lógica estándar descrita en el código SQL entregado).

V. PARTE 2: CREACIÓN DE DISPARADORES Y GESTIÓN DE USUARIOS

A. TRIGGERS

1. Trigger de Auditoría

Este mecanismo registra cualquier modificación en las tablas críticas para mantener la trazabilidad de los datos.

- **Tabla de Auditoría:**
- CREATE TABLE IF NOT EXISTS pl1final.auditoria_eventos (
- evento_id SERIAL PRIMARY KEY,
- tabla_afectada TEXT NOT NULL,
- tipo_evento CHAR(1) NOT NULL, -- 'I', 'U', 'D'
- usuario_db TEXT NOT NULL,
- fecha_hora_evento TIMESTAMP DEFAULT current_timestamp,
- datos_antiguos JSONB, -- Estado anterior (para Update/Delete)
- datos_nuevos JSONB -- Estado nuevo (para Insert/Update)
-);
- Función y Trigger:

Se implementa una función genérica pl1final.func_trigger_auditoria() que detecta la operación (TG_OP) e inserta el registro correspondiente.

```
CREATE TRIGGER trig_auditoria_drivers
```

```
AFTER INSERT OR UPDATE OR DELETE ON pl1final.drivers
```

```
FOR EACH ROW EXECUTE FUNCTION pl1final.func_trigger_auditoria();
```

- **Prueba de Funcionamiento:** Al ejecutar INSERT INTO pl1final.drivers ..., se verifica que aparece automáticamente una nueva fila en auditoria_eventos con el usuario responsable y los datos insertados en formato JSON.

2. Trigger de Puntuación Total

Mantiene una tabla desnormalizada con la suma de puntos para optimizar consultas de clasificación histórica.

- **Tabla de Puntos:** CREATE TABLE pl1final.puntuacion_total_pilotos (driverRef TEXT PK, puntos_totales FLOAT);

- **Trigger:** Se dispara AFTER INSERT en la tabla results. Utiliza una lógica "Upsert" (INSERT ... ON CONFLICT DO UPDATE): si el piloto ya existe, suma los puntos nuevos a los existentes; si no, crea el registro.

B. CREACIÓN Y PERMISOS DE USUARIOS

Se han creado roles para gestionar los permisos de forma eficiente y escalable:

1. **ADMINISTRADOR (rol_admin_f1):** Control total.
 - GRANT ALL PRIVILEGES ON SCHEMA pl1final TO rol_admin_f1;
2. **GESTOR (rol_gestor_f1):** Operativa diaria (CRUD), sin modificar estructura.
 - GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES...
 - Permiso sobre secuencias para permitir inserciones en tablas con ID autoincremental.
3. **ANALISTA (rol_analista_f1):** Solo lectura global.
 - GRANT SELECT ON ALL TABLES...
4. **INVITADO (rol_invitado_f1):** Lectura restringida.
 - Permiso explícito: GRANT SELECT ON results, drivers, races...
 - Permiso explícito sobre la Vista: GRANT SELECT ON pl1final.VistaPuntosPorTemporada...
 - Prohibición explícita: REVOKE SELECT ON lap_times, pit_stops...

Prueba de Permisos (Invitado):

Al intentar ejecutar la Consulta 9 (que accede a lap_times) con el usuario invitado desde la interfaz Python, la base de datos devuelve el error: ERROR: permiso denegado a la tabla lap_times. El programa captura este error y muestra un mensaje explicativo, confirmando que la seguridad funciona correctamente.

VI. PARTE 3: CONEXIÓN CON PROGRAMAS EXTERNOS (PYTHON)

Explicación de la Conexión

Se ha utilizado la librería psycopg2 por su robustez y cumplimiento con el estándar DB-API 2.0. La conexión se gestiona mediante la función psycopg2.connect(), desactivando el autocommit (conn.autocommit = False) para permitir el manejo transaccional manual (commits y rollbacks explícitos) en operaciones de escritura, garantizando la integridad de los datos.

Código Fuente (Fragmento Representativo)

El siguiente código muestra la función de conexión y la ejecución de una consulta, incluyendo el manejo de excepciones para permisos denegados:

```
import psycopg2

from getpass import getpass

def connect_to_db(user, password, host="localhost", dbname="f1"):
    try:
        # Conexión sin autocommit para control transaccional

        conn = psycopg2.connect(dbname=dbname, user=user, password=password,
                                host=host)

        conn.autocommit = False

        return conn
    except psycopg2.Error as e:
        print(f"Error de conexión: {e}")
        return None

def ejecutar_consulta_segura(conn):
    # Consulta 9: Ejemplo de consulta que podría fallar por permisos

    sql = """

        SELECT d.forename, d.surname, lt."time"

        FROM pl1final.lap_times lt

        JOIN pl1final.drivers d ON lt.driverRef = d.driverRef

        WHERE lt."time" = (SELECT MIN("time") FROM pl1final.lap_times);

    """

    try:
        with conn.cursor() as cur:
            cur.execute(sql)

            rows = cur.fetchall()

            # ... impresión de resultados ...
```

except psycopg2.Error as e:

```
# Captura específica de errores de BD (como permisos denegados)
```

```
print(f"Error al ejecutar consulta: {e}")
```

```
print("-> Es posible que su usuario no tenga permisos para esta acción.")
```

```
conn.rollback() # Importante: Resetear la transacción fallida
```

El programa ofrece un menú interactivo que adapta las opciones disponibles (Insertar Datos) según el rol del usuario conectado (solo Admin y Gestor pueden ver las opciones de escritura).

VII. PROBLEMAS ENCONTRADOS Y SOLUCIÓN ADOPTADA

1. **Permisos sobre Vistas y Dependencias:** Se identificó que el usuario invitado tenía permiso de lectura sobre las tablas base pero fallaba al consultar la Vista creada en la Parte 1 si esta no se había creado explícitamente antes de asignar los permisos. **Solución:** Se integró la sentencia CREATE OR REPLACE VIEW dentro del script de seguridad (Triggers.sql) justo antes de la asignación de permisos (GRANT), asegurando que el objeto exista y evitando errores de dependencia.
2. **Codificación de Caracteres en Windows:** Se detectaron problemas (UnicodeDecodeError) al importar scripts SQL con caracteres latinos (tildes) en entornos Windows con codificación por defecto WIN1252. **Solución:** Se aseguró el guardado de todos los archivos SQL en formato **UTF-8** y se configuró el cliente psql y la conexión Python para trabajar explícitamente con esta codificación.